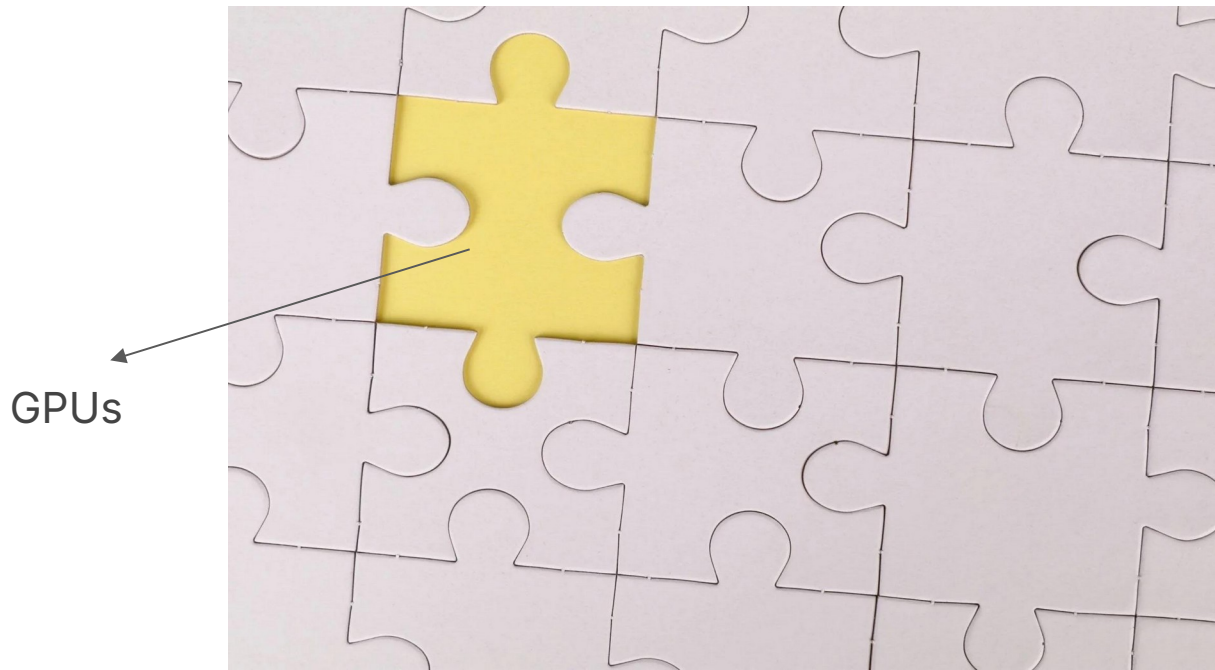


GPUs and Inference Engineering

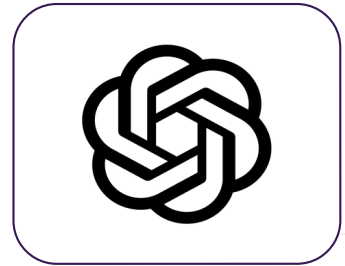
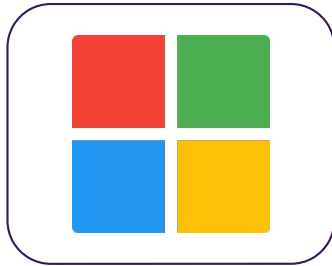
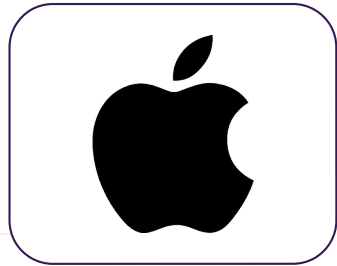
Making AI Models Work in the Real World

The Final Piece of the Puzzle

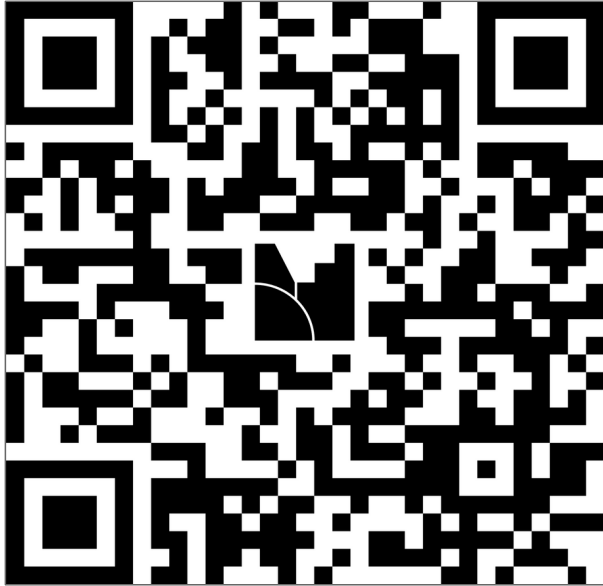
Connecting everything you've learnt to how AI runs in the real world



What is the most valuable company in the world today (in terms of market capitalization)?



Join the Mentimeter Poll

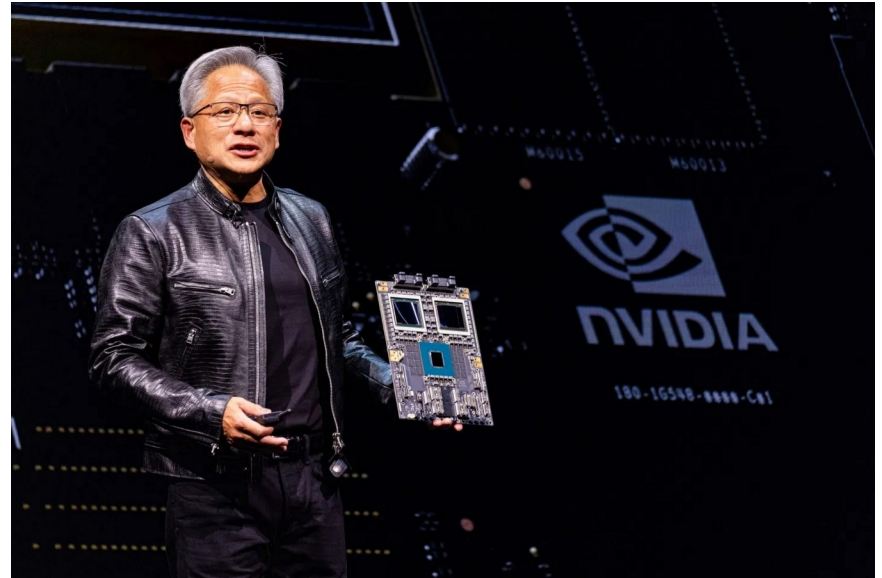


Or go to <https://menti.com>

And enter the code: **21159839**



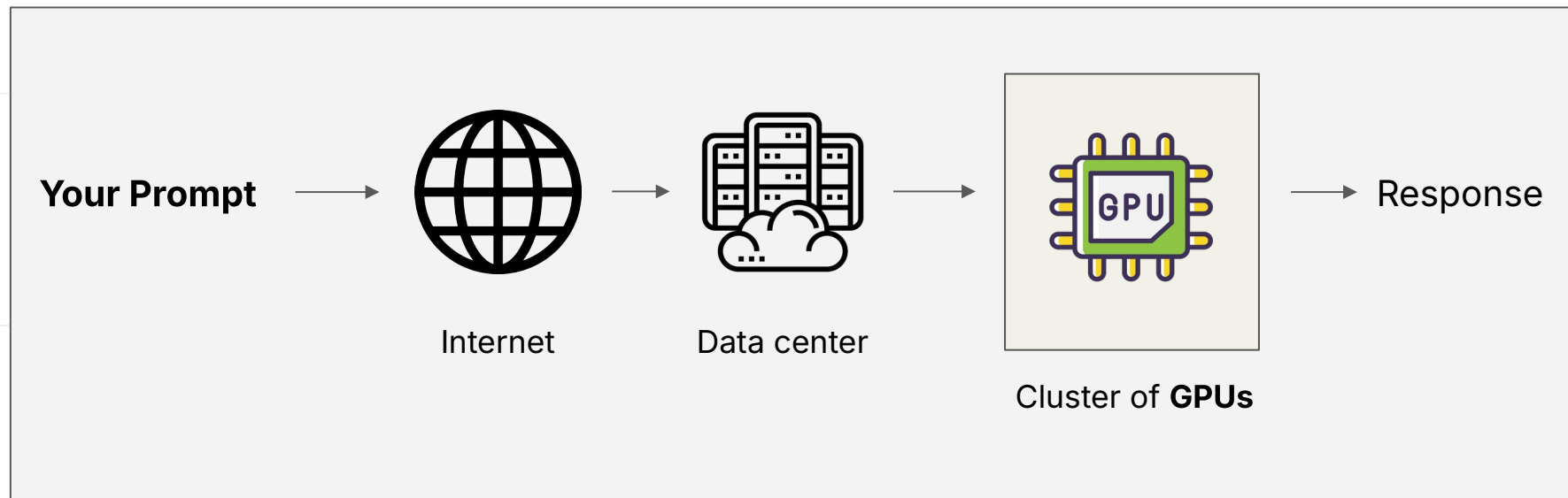
Market Capitalization: ~\$5.3
Trillion+ (as of August 2026)



Because they make **GPUs**, the hardware powering modern AI systems.

GPUs are so important!

Every AI application you use today runs on GPUs. ChatGPT, Google Gemini, image generation, recommendation systems, self-driving cars, and even climate models all depend on GPUs.



The Modern AI Stack

AI does not run on models alone.

**GPUs, Compute, Energy, Data
Centers, Connectivity**



Meet Ben!

Does not know why GPUs
matter for modern AI

Does not know how to measure
compute and memory

Does not know how to run
experiments on GPUs



Memorized "Attention is All You Need"
but cannot build something with it

Does not understand the cost of
training and running AI models

Does not know how to optimise
GPU usage

Don't Be Like Ben ✖

This lecture will show you how.



Before we get started...



 **This is important, and it will be fun!**

Everything today connects to how we **train and deploy AI models in the real world.**

 **Attention is all you need**

Focus on the **key intuitions**, not memorizing every formula or technical detail. **Ask questions too!**

 **You also need a pen + notebook, maybe a calculator too**

There will be **questions, quick calculations, and challenges** along the way.

01. The GPU

What is a GPU, and why does it matter for modern AI?



The CPU

- 🧠 **Brain of the computer** that executes instructions
- **Optimised for general-purpose tasks**
- **Few powerful cores** that handle complex operations sequentially

Examples: Intel Core i5, AMD Ryzen, Apple M-series





If you need to solve 1,000,000 simple arithmetic problems (e.g., $2 \times 8 + 5$), who do you hire?

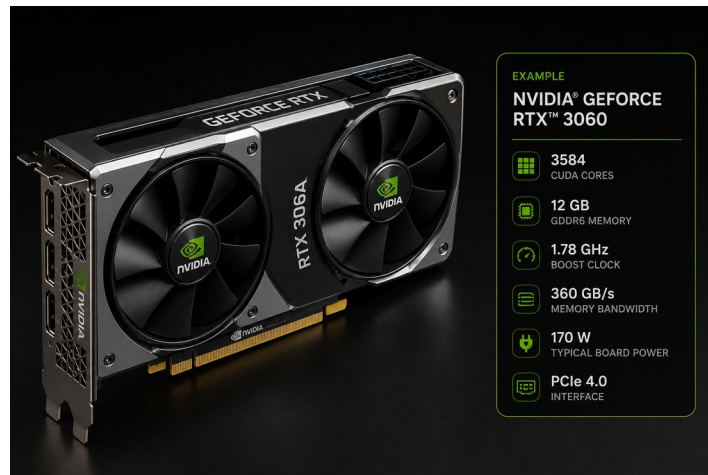
- **Option A:** 4 world-class mathematicians
- **Option B:** 1000 high-school students

Assume a world-class mathematician solves at a speed of 1 second per problem while a high-school student spends 3 seconds per problem.

Now The GPU

GPU → Graphics Processing Unit (The “Workhorse” of modern AI)

- **Built with thousands of smaller, specialised cores.** Think of this as a giant army of simple workers.
- **Designed for massive parallel processing,** performing millions of calculations simultaneously.
- **Originally built for graphics,** but now powers modern AI.



Examples: NVIDIA GeForce RTX, Tesla (e.g., T4), Ampere, Hopper, Blackwell families

GPUs Are Very Important!

Think About GPT-3 (the breakthrough model that paved the way for ChatGPT)

- GPT-3 has **~175 billion parameters**
- Training on a single CPU would be impractical or take years!
- Inference on a CPU would be extremely slow



The massive size of modern AI models makes GPUs a necessity, not just a luxury.

So When Do You Need a GPU?

1. Analyze 100,000 rows of sales data and calculate summary statistics



CPU

2. Train a neural network with 20 million parameters



CPU or GPU (depends on the data, architecture, training time requirements, etc.)

3. Train a 100-billion-parameter language model on a massive text dataset

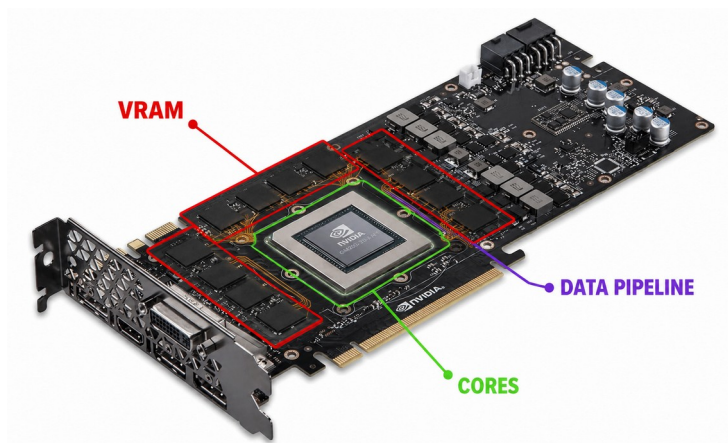


GPU (many of them)

A GPU helps when you have huge amounts of the same math happening in parallel.

Inside the GPU System

The key components that make a GPU work



- **Compute Cores ("The Machines/Workers")** → perform calculations simultaneously
- **VRAM ("The Warehouse")** → memory for storing model parameters, activations, and intermediate results.
- **Data Pipeline ("The Conveyor Belt")** → moves data between memory and compute

Modern AI is powered by massive parallel computation, fast storage, and high speed data movement.

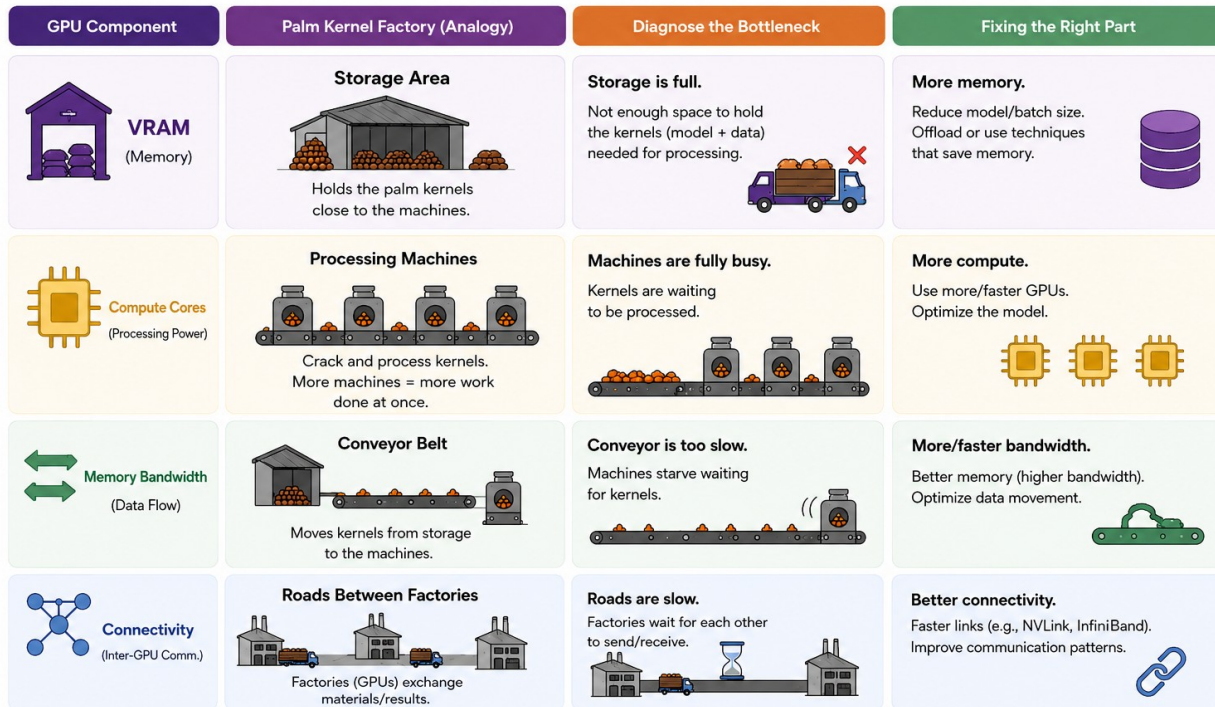
GPUs Can Have Bottlenecks

Any one of its three main components can limit performance

Component	Bottleneck
VRAM	The model doesn't fit. There isn't enough memory to load it.
Compute Cores	The GPU can't compute any faster. The cores are already fully utilized.
Data Pipeline	The cores are waiting for data. Data can't be delivered fast enough from memory.

A GPU is only as fast as its bottleneck. For a GPU to work well, all components must work together efficiently.

Palm Kernel Processing Factory ⇔ GPU System



Diagnose the bottleneck:
Where is the system slowing down?



Find the root cause:
Which part is limiting performance?



Fix the right part:
Improve only what's needed.

To get the best performance out of our GPUs, we need to be able to diagnose and remove these bottlenecks.

But first, let's measure what our model needs and determine what kind of GPU we need.

02. Measuring Compute & Memory

How much compute and memory does my model need, and what will it cost?



Scaling Laws

Why does bigger AI keep getting better?

As we scale models, data, and compute, model performance tends to improve predictably.

Why this matters more than ever:

Bigger models

→ more compute to train them

More data

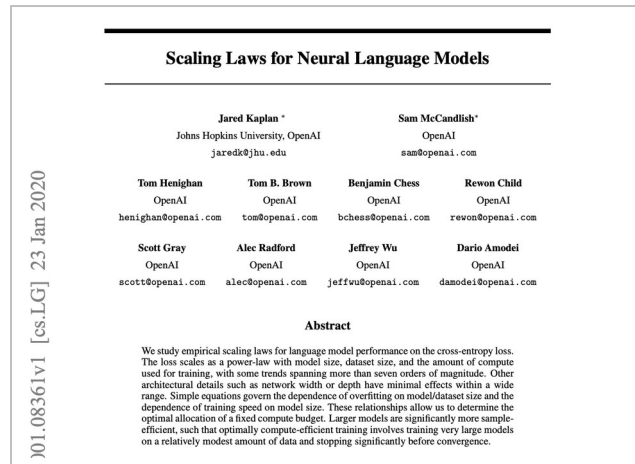
→ more compute to process it

Better models

→ more demand for efficient hardware

Scaling AI

→ efficiency becomes a research problem, not just a hardware problem



Kaplan et al., 2020

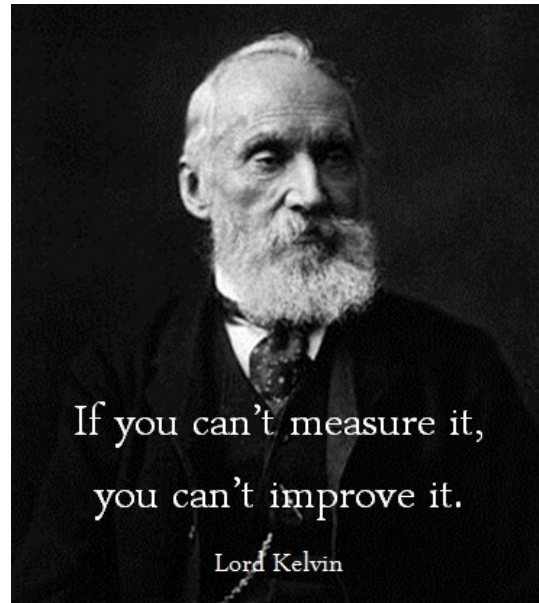
As AI scales, the cost of intelligence scales with it.

Why Measuring **Compute** and **Memory** Matters?

Before training or running AI models using GPUs, we first need to ask:

- How much **compute** does my model require?
- How much **memory** does my model require?

And ultimately: What kind of GPU do I need?



Measuring Compute

- "Compute" is simply the actual mathematical work performed by the computer.
- Remember, modern AI relies almost entirely on massive **matrix multiplication**.
- This involves performing billions, or even trillions, of simple mathematical operations (like adding and multiplying numbers).
- So we measure compute by counting the **total number of mathematical operations** required.
- Measuring compute helps us choose the right GPU based on requirements and identify compute bottlenecks.

How would you charge me (💰) if I asked you to multiply two matrices for me?

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \times \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} = \begin{bmatrix} 7 \\ 15 \end{bmatrix}$$

Calculations for the result vector elements:

- Top element (7): $1 \times 1 + 2 \times 3$
- Bottom element (15): $3 \times 1 + 4 \times 3$
- Top element (10): $1 \times 2 + 2 \times 4$
- Bottom element (22): $3 \times 2 + 4 \times 4$

Can you count the **number of mathematical operations** performed to compute the result? **Can you see how these operations can be parallelised?**

To measure compute in modern AI, we count the number of floating-point operations (FLOPs) .

Because the numbers used in AI models, such as model parameters, are typically represented as floating-point numbers.

But we also need to measure **how quickly the GPU can do the task?**

How We Measure Compute: FLOPs vs. FLOPS

FLOPs (Floating-Point Operations) → The Workload

The total number of floating-point operations needed to complete a specific task (like training a model or generating a response).

FLOPS (Floating-Point Operations Per Second) → The Speed

This is a measure of how many operations the GPU can perform every single second. Think of this as how fast the GPU can perform the work.

The Rule: Assuming 100% utilization, the total FLOPs required divided by the FLOPS of the GPU gives you the total time the task will take.

Estimating Compute for a Transformer Model

How many FLOPs do you need to train a transformer model?

$$\text{FLOPs} \approx 6 \times N \times D$$

Where:

- **N** = Number of parameters
- **D** = Number of training tokens (data the model is trained on)
- **Why 6?**

The compute required scales linearly. If you double the parameters or double the data, you double the compute.



Worked Example

You want to train a 1 billion (1B) parameter LLM on 100 billion (100B) tokens for your research project using a GPU with 100 TeraFLOPS

- Estimate the total FLOPs required (**Hint:** use the $6 \times N \times D$ formula)
- How long will it take to train (in hours)?

Recall: 1 billion = 1×10^9 , 100 billion = 100×10^9 , 100 TeraFLOPS = 100×10^{12} FLOPS. Assume **Utilization** (how much of the theoretical GPU performance you actually achieve) is 100%.



Money Matters: How Much Does Training Cost?

Every hour of GPU compute comes with a cost. For a single GPU:

$$\text{Training Cost} \approx \text{GPU Hours} \times \text{Cost Per GPU Hour}$$

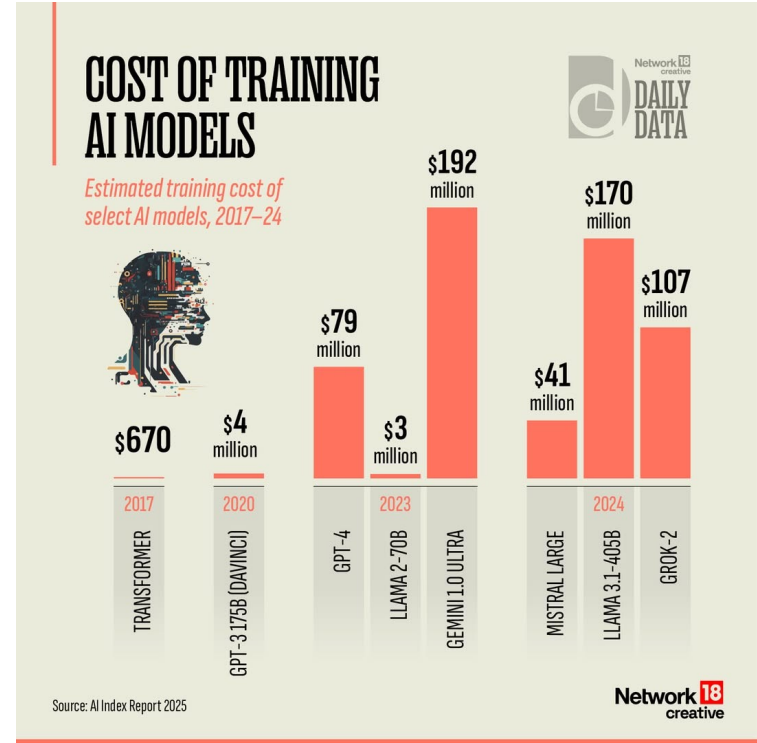
For our example, if the GPU costs **\$2/GPU-hour**:

$$\text{Training Cost} = 1,667 \text{ hours} \times \$2 = \text{\$3,334}$$

Compute is not free! More training FLOPs = More Time (GPU Hours) = More Money.

No Free Lunch

Training large AI models comes at a huge compute cost



Measuring Memory

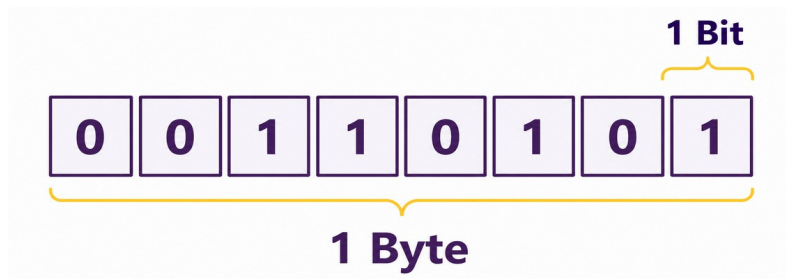
- The model and data currently being processed must fit into the GPU memory (VRAM).
- If your model doesn't fit entirely inside the VRAM, it simply won't run. It crashes.
- Measuring memory helps us identify memory bottlenecks and choose the right GPU based on our requirements.

Compute determines how fast your model does the task, but memory determines if it can do it at all. **More memory = Ability to fit larger models and more data.**

How Computers Store Numbers

Model parameters are just numbers

- Every single parameter in an AI model is ultimately just a number that must be physically stored inside the GPU's memory.
- Computers represent all numbers using binary (0s and 1s). A **bit** is the smallest unit (0 or 1).
- We measure the memory required to store a number using **bytes**. A byte = 8 bits



The way we represent a number determines how much memory it requires.

Precision: More Bits, More Accuracy

The precision with which numbers are stored determines their GPU memory usage.

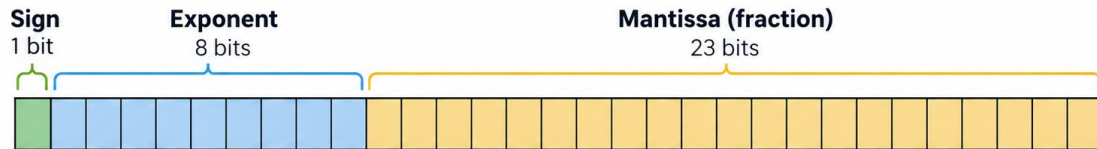
Precision	Size (Bytes)	Description	Example (Storing π)
float32 (FP32)	4 Bytes	Traditional floating-point format. High precision.	3.1415927
float16 (FP16)	2 Bytes	Uses half the memory of FP32.	3.1406
bfloat16	2 Bytes	Designed for AI. Keeps the same numeric range as FP32 but with lower precision.	3.1406 (approx.)
int8	1 Byte	8-bit integer format i.e. no decimal numbers.	3

The Trade-off: Using fewer bytes (lower precision) saves massive amounts of GPU memory, but we lose mathematical accuracy.

Why is **bfloat16** so common?

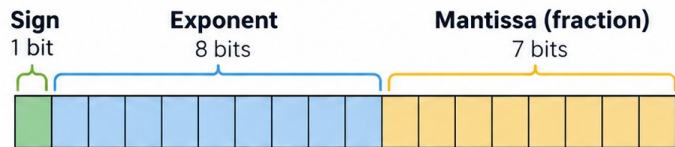
FP32

32 bits



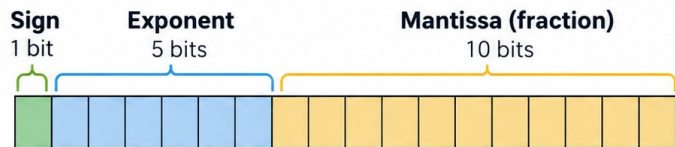
BF16

16 bits



FP16

16 bits



bfloat16 gives the wide numerical range of FP32, while using half the memory. Developed by Google Brain.

What Fills GPU Memory

But first, what happens in the GPU when working with neural networks?

Training

- Input data passes through the model (forward pass)
- Predictions are compared to true labels
- An error (loss) is computed
- The model computes gradients (backpropagation)
- Parameters are updated

Inference

- Input data passes through the model (forward pass)
- A prediction is produced
- No parameters are updated

Training a model requires far more compute and memory than running it (inference). **Why?**

What Happens During Training

1

Load a mini-
batch

A group of training examples is fed into the model

2

Forward pass

The model computes the predictions and the loss.

It also stores **intermediate activations** needed later for backpropagation.

3

Backpropagation

Using the loss, it computes **gradients** for the parameters.

4

Optimizer update

The optimizer (e.g. Adam) uses those gradients, plus any **optimizer states**, to update the parameters.

1 Noisy gradient descent

reacts to every noisy gradient



2 Momentum (1st moment)

keeps moving in the consistent downhill direction



3 Adam = momentum + variance-aware step size

adapts step size using gradient variance

fast + stable

smaller steps on steep / noisy directions

larger steps on flat / stable directions



Noisy / bumpy terrain



Smooth / consistent slope



Step (update)



Step size (small $\rightarrow$ large)

What Happens During Inference

1

Input

Input is sent to the model
for processing.

2

Forward pass

Input sequence is passed
through the model to generate
a prediction. Activations may
be temporarily stored.

What Fills GPU Memory

Training

Learning from data to update the model's parameters

Parameters

Activations

Gradients (~1x parameters)

Optimizer states (~2x parameters)

Inference

Running the model to make predictions

Parameters

Temporary activations

Plenty of room left

Estimating Memory is Simple

We can directly estimate how much GPU memory is required to store a model's parameters

$$\text{Parameter Memory} = N \times B$$

Where:

- **N** = Number of parameters
- **B** = Bytes per parameter
- **Example:** A 1 billion parameter model stored in FP16 (2 bytes per parameter) requires $10^9 \times 2$ bytes = 2 GB

More parameters → More memory, Higher precision → More memory

How Do We Estimate Memory For Training and Inference?

1. Inference Memory

Inference Memory $\approx$ Parameter Memory + Temporary Activation Memory

2. Training Memory

Training Memory $\approx$ Parameter Memory + Gradient Memory + Optimizer State Memory + Activation Memory

GPU Memory for Inference



If the VRAM was
a backpack

GPU Memory for Training





Worked Example

You have a 16 GB NVIDIA GPU and want to run inference and train an LLM with 8 billion (8B) parameters stored in bfloat16.

Will the model fit in GPU memory for:

- Training? (*Assume Activation Memory = 10 GB, Adam optimizer*)
- Inference?

Recall: bfloat16 = 2 bytes per parameter

What Eats Your GPU Memory

- **More parameters** → more memory during training and inference; during training, also more gradients and optimizer states
- **Larger batch size** → more sequences (e.g. sentences) processed at once → more activations stored during training
- **Longer sequence length** → more tokens to process during training and inference

03. GPU Performance Bottlenecks

What is really limiting my GPU performance?



What Is Limiting My GPU?

Four common ways a GPU gets held back



Compute-bound

Held back by
Calculations



Memory-bound

Held back by
VRAM capacity



Memory-bandwidth-bound

Held back by
Data movement
speed



Communication-bound

Held back by
Communication
between GPUs

Before optimising, identify the bottleneck.

Think About Attention

What kind of bottleneck does Attention have?

- A Compute-bound
- B Memory-bound
- C Memory-bandwidth-bound
- D Communication-bound



Think About Attention

What kind of bottleneck does Attention have?

A Compute-bound

B Memory-bound

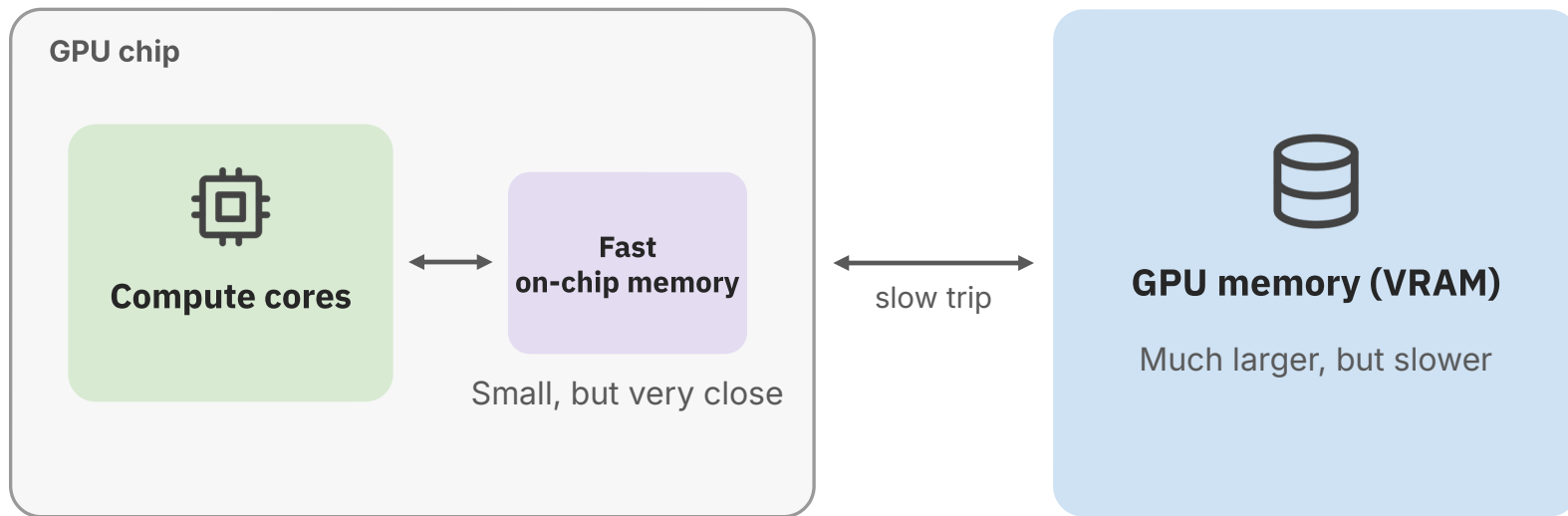
C Memory-bandwidth-bound ✓

D Communication-bound



Answer: C. Memory-bandwidth-bound. But why?

A GPU Has Two Kinds of Memory

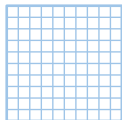


Small and fast. Big and slower.

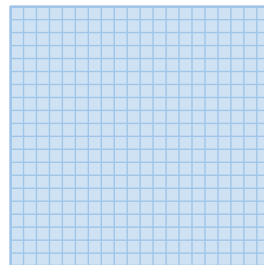
Attention Builds a Big Table

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

QK^T builds a table: one score for every pair of words



10 words
small table

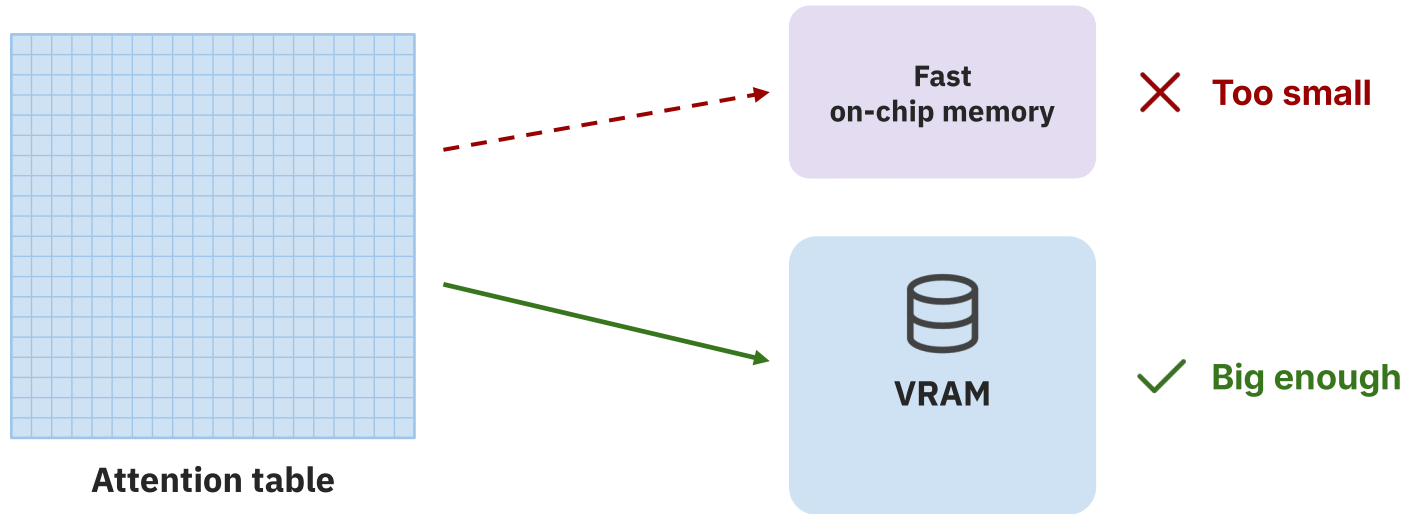


1,000 words
huge table

More words means a much bigger table.

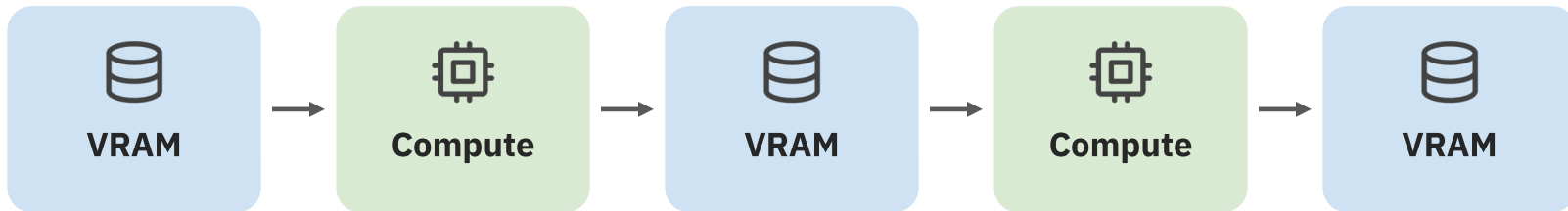
The Table Doesn't Fit on the Chip

Too large for fast on-chip memory, so the GPU keeps it in VRAM



When data doesn't fit in fast memory, we have to use slower memory.

Back and Forth, Again and Again



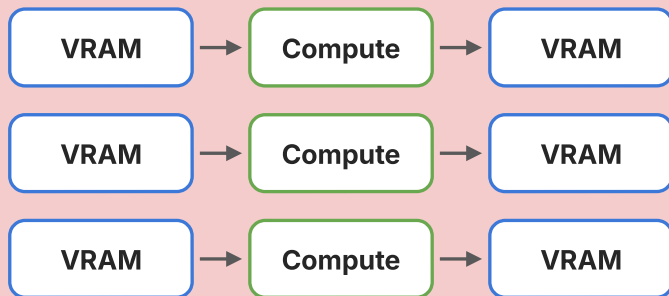
The GPU repeatedly moves data between VRAM and the compute cores.

The calculations may be fast, but moving the data takes time.

When data movement becomes the bottleneck, we are memory-bandwidth-bound.

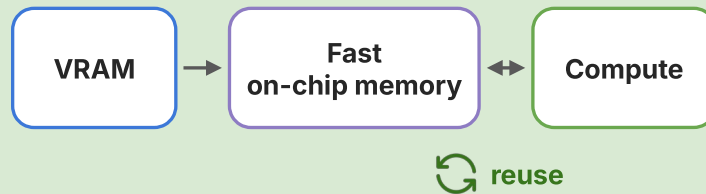
FlashAttention: Move Less Data

Traditional Attention



Many slow trips to VRAM

FlashAttention



Works on small pieces in fast on-chip memory

Far fewer trips to VRAM

Same maths. Less data movement.



**If data movement is the problem,
move less data.**

We identified the bottleneck. **Now, how do we optimise GPU workloads?**

04. GPU Optimization Techniques

How can I get the most out of my GPU resources without sacrificing performance?



Why Do We Need to Optimize?



More parameters

Often better model capability



Larger batch size

More stable training

💰 But **GPU memory is limited**, and GPUs are expensive.

So how do we do more with less GPU memory?

Your GPU Memory Is a Backpack



✗ Full



Buy a bigger bag

A GPU with more VRAM. Expensive.



Pack smarter

Optimization on a single GPU



Share the load

Use multiple GPUs

First, let's learn to pack smarter.

Three Tricks to Pack Smarter

1

Quantization

Reduce the size of the items.

Carry small paperbacks instead of large hardcovers.



2

Mixed-Precision Training

Use a mix of small and big items.

Important files in full-size folders, the rest in compact ones.



3

Gradient Accumulation

Make multiple trips.

Carry smaller loads, several times.



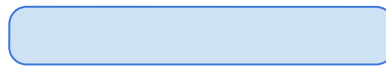
Same bag. Smarter packing.

Our Brains Naturally “Quantize”

To save mental storage space, we compress complex information into simpler forms

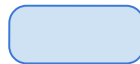


$\pi \approx 3.14159265$



Exact, more space

$\pi \approx 3.14$



Rounded, less space

Rounding saves space, but adds a small error.

Quantized LLMs do the same: drop the “decimals” to save GPU memory, with a tiny cost to accuracy.

Quantization

Smaller numbers, smaller memory

What it is

Convert parameters from high precision to lower precision

Example

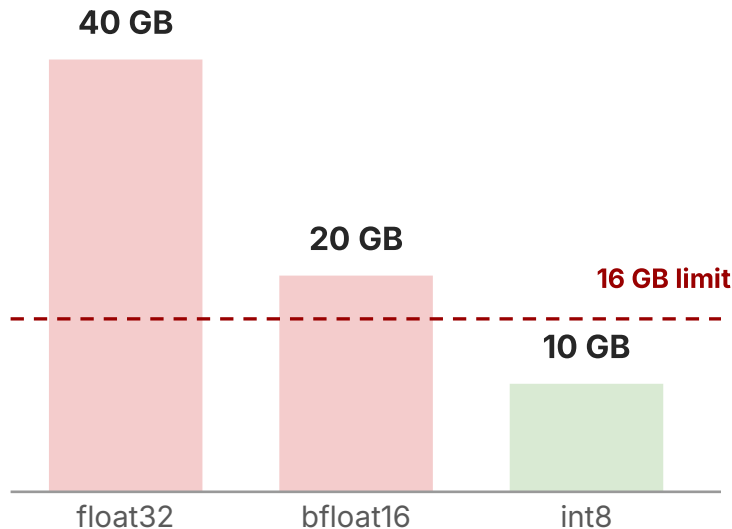
Converting a model from bfloat16 to int8

Benefit

Uses much less GPU memory and runs faster

Trade-off

Can cause a minor drop in accuracy



Can you fit a 10 billion parameter model on a Google Colab T4 GPU with 16 GB VRAM?

What About Training?

Quantization works for inference. Do you think it can work during training too?

Yes, but we need to be careful. Training makes tiny, precise adjustments. Round too much (like int8 or int4) and those updates drop to zero.

$$1.00 + 0.0001 = 1.00$$

✗ **The model stops learning**



So for training, we mix precisions.

Mixed-Precision Training

Getting the benefits of lower precision during training

What it is

Train with a mix of high- and low-precision formats

Benefit

Lower memory and faster training, with little or no loss in accuracy

Trade-off

Needs hardware that supports mixed precision

bfloat16

Forward pass through the model
attention, matrix multiplications

float32

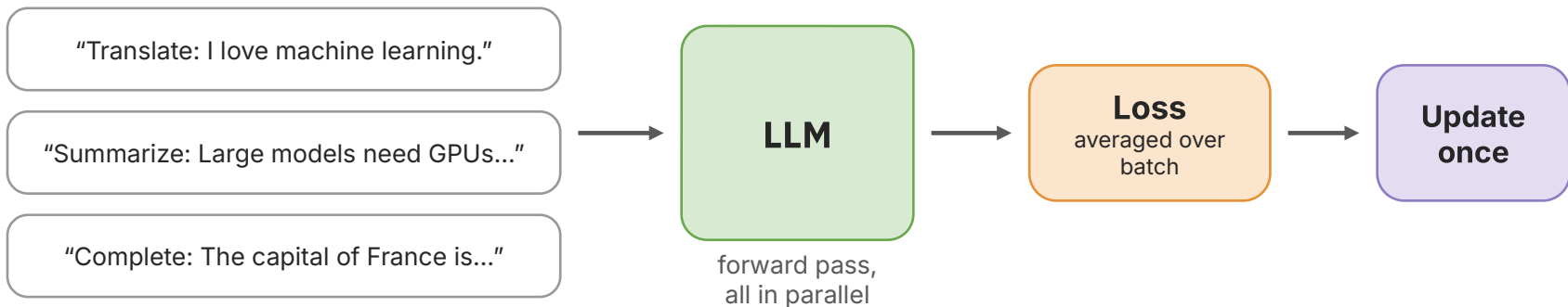
Loss computation, optimizer updates

Most computation runs in bfloat16 for efficiency. Loss and weight updates stay in float32 for stability.

What Is a Batch?

A group of examples processed together before each weight update

Batch with 3 examples



The model updates its weights once per batch.

Large batch sizes improve training stability, but they often exceed the available GPU memory.

What if you could get the stability of a large batch with the memory of a small batch?



Gradient Accumulation

A big batch doesn't fit. So split it, accumulate, then update.

What it is

Train with a large effective batch using several small batches

How

Add up gradients from each small batch, then update the weights

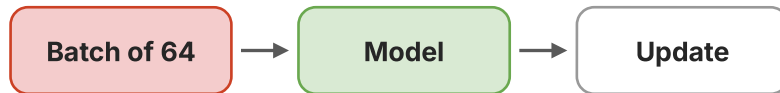
Benefit

Big-batch stability with less GPU memory

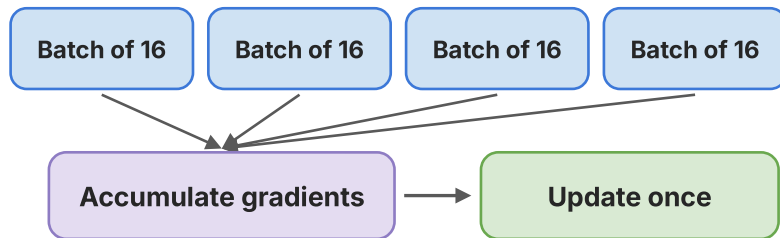
Trade-off

Slower: each update needs several passes

Instead of one large batch...

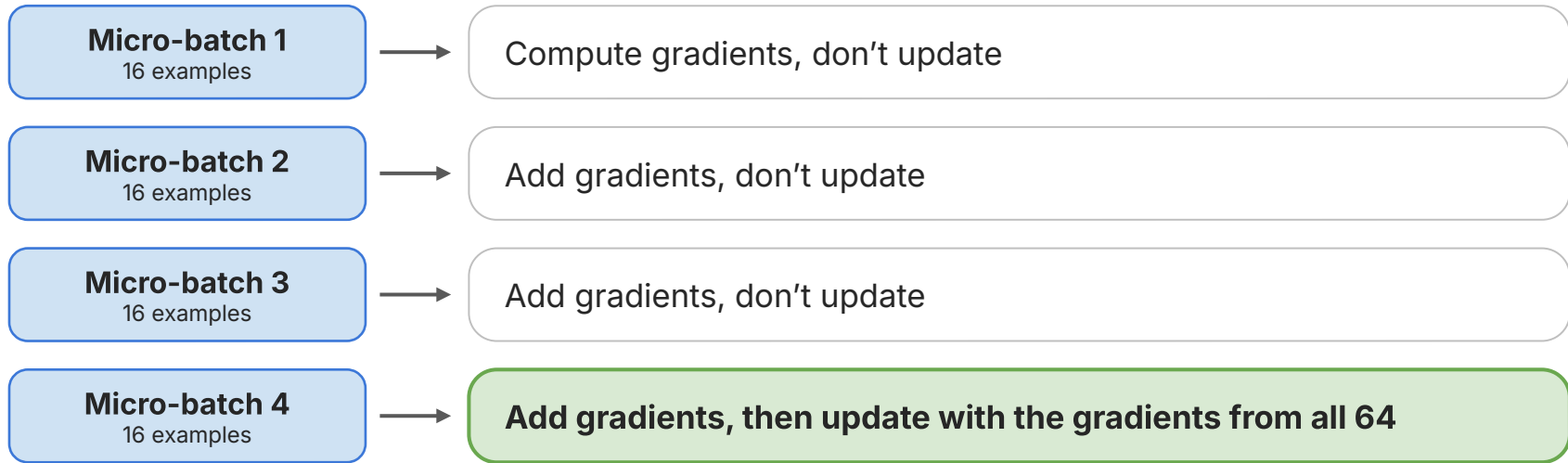


...we use several small batches



Gradient Accumulation: An Example

You want a batch of 64, but your GPU only fits 16 examples at once



Four small steps. One big-batch update.

Sometimes, One Bag Is Not Enough



One GPU: too much for one bag



GPU 1



GPU 2



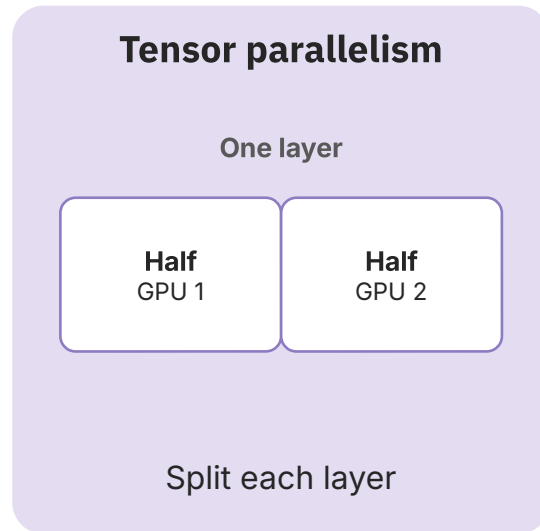
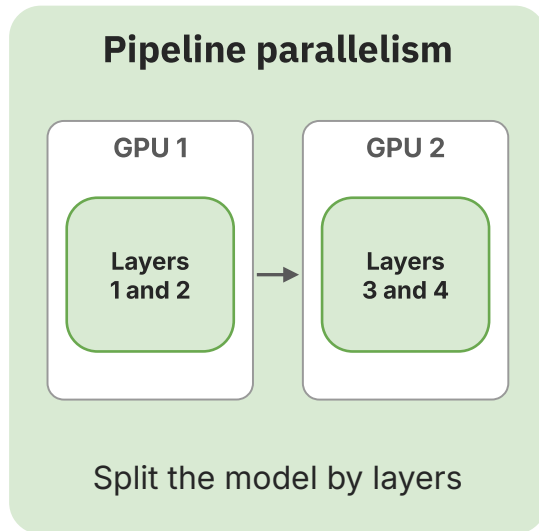
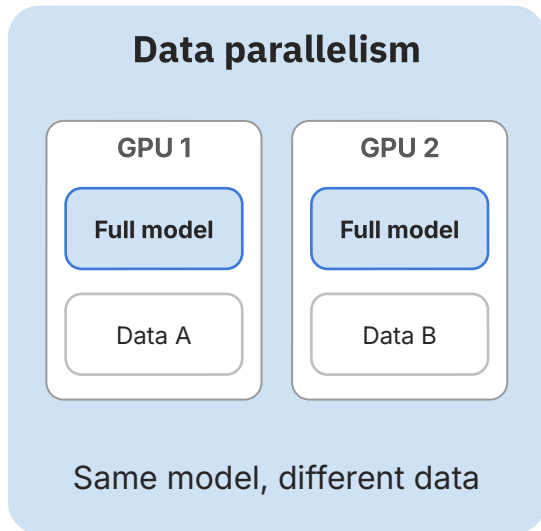
GPU 3

Many GPUs: share the load

Multi-GPU training: share the work across several GPUs.

Multi-GPU Training

Three ways to share the load



More GPUs means more communication. **Now we can become communication-bound.**

What Would You Do?

1

You need to run a 7B model on a laptop with 16 GB of memory. It doesn't fit.



Quantization

2

You want to train faster and use less memory, without losing numerical stability.



Mixed-Precision Training

3

You want a batch size of 64, but your GPU only fits 8 examples at a time.



Gradient Accumulation

4

Your model is too big for one GPU, even after all these tricks.



Multi-GPU Training

05. Inference Engineering

What really happens when we serve a model to users?



Why Inference Engineering Matters

Training happens once. Inference happens every time someone uses the model.



Training

Happens once, or a few times
A huge, one-off cost



Inference

Happens with every user request
Millions of requests, every day

For popular models, inference can cost more than training over the model's lifetime.

What Inference Engineers Optimize

Making LLMs fast, cheap, and practical to deploy in the real world



Latency

How fast each token arrives



Throughput

How many users served at once



Cost

Price per request or per token



Memory

Model weights plus the KV cache

Tools you already know:

Quantization

FlashAttention

Batching

KV cache

Multi-GPU serving

Inference engineering is where GPU optimization meets real users.

05. Using GPUs Responsibly

Hmmm...what are the environmental impacts of using GPUs?



GPU & Model Acceleration Activity

Work in pairs! Explain and reflect on the implications and trade-offs of your decisions.

1

Calculate training memory and determine which GPUs can fit the model.

2

Compare training time and cost across GPU options.

3

Evaluate inference memory and the impact of different precision levels.

4

Make and justify a final GPU and optimisation decision.

What Next?



GPU Optimisation

Research ways to make AI models faster, cheaper, and more memory-efficient on GPUs.



Your AIMS Project

Apply these concepts when choosing models, estimating compute requirements, and designing experiments.



Inference Engineering

One of the most in-demand roles in AI today. Make LLMs efficient and practical to deploy in the real world.



AI Systems Research

Explore how hardware, models, and algorithms work together to build efficient AI systems.

Resources

Keep exploring GPUs and inference engineering



Inside an AI Data Center

kiankyars.github.io/gigawatt



Inference Engineering Book by Philip Kiely

baseten.co/inference-engineering



Inference Engineering Book (Interactive)

inferenceengineering.tech/exercises



NVIDIA GPU Programming Guide

developer.nvidia.com/nvidia-gpu-programming-guide



AIMS GCP Tutorial

rexsimiloluwah.github.io/gcp-tutorial-aims